GLO-2005 Modèles et langages des bases de données pour ingénieurs

Langage SQL : Les astuces

Survol

- Création, mise à jour, suppression des relations et tuples
- Recherches avancées dans les relations
- Jointure
- Vues
- Livres
  - Ramakrishnan & Gehrke : sections 5.1 à 5.6
  - Connolly & Begg : chapitre 6, sections 7.1, 7.3, 7.4

Création de relations

```
CREATE [TEMPORARY] TABLE [IF NOT EXIST] nom
{ ( nom_attribut domaine [contrainte_attribut] [,…],
            [contraintes_relation [,…]] )
| LIKE ancien_nom | [AS requête] };
```

- On a vu la forme de base
  - `CREATE TABLE Etudiants(idul char(5), nom char(40), age integer, moyenne double, nas integer);`
- On peut créer une relation de forme identique à une qui existe
  - `CREATE TABLE nouveauxEtudiants LIKE Etudiants;`
- Créer deux relations avec le même nom cause une erreur
  - On peut l'éviter en incluant une condition
  - `CREATE TABLE IF NOT EXISTS Etudiants;`

Insertion par requête

```
INSERT INTO nom_relation { [( nom_attribut [,…] )]
VALUES (valeurs) [,…] | requête };
```

- On a vu l'insertion de tuples un par un (première partie de la requête)
  - `INSERT INTO Etudiants(idul,nom,age,moyenne,nas) VALUES ('DIOPQ','Diane',26,3.3,456789012);`
- On peut insérer les tuples retournés par une requête sur une autre relation
  - Suppose que la relation cible a les bons attributs
- `INSERT INTO` *nom_relation requête;*

Insertion par requête

- `CREATE TABLE Etudiants22 LIKE Etudiants;`

- `INSERT INTO Etudiants22`
  `SELECT *`
  `FROM Etudiants`
  `WHERE age > 22;`

Etudiants

idul	nom	age	moyenne	nas
ALXYZ	Alice	24	4.1	123456789
BOUVW	Bob	27	3.6	2345
CERST	Cédric	22	3.1	3456

Etudiants22

idul	nom	age	moyenne	nas
ALXYZ	Alice	24	4.1	123456789
BOUVW	Bob	27	3.6	234567890

Création et insertion par requête

- On peut le faire lors de la création de la relation
-
```
CREATE TABLE Etudiants22 AS
SELECT *
FROM Etudiants
WHERE age > 22;
```

Modification de relations

```
ALTER TABLE nom {
      ADD COLUMN nom_attribut domaine [contrainte_attribut]
|     ADD [INDEX|KEY] nom propriétés
|     DROP COLUMN nom_attribut
|     DROP [INDEX|KEY] nom };
```

- On peut ajouter un attribut à une relation
  - `ALTER TABLE Etudiants`
    `ADD COLUMN adresse char(100);`
  - S'il y a déjà des tuples dans la relation, ils reçoivent un nouvel attribut avec la valeur `null`
- On peut retirer un attribut d'une relation
  - `ALTER TABLE Etudiants`
    `DROP COLUMN moyenne;`
  - S'il y a déjà des tuples dans la relation, ils perdent l'attribut et l'information contenue est perdue

Suppression de relations

```
DROP [TEMPORARY] TABLE [IF EXISTS] nom ;
```

- On peut supprimer une relation
  - `DROP TABLE Etudiants;`
  - Toute l'information contenue dans la relation est perdue
- Supprimer une relation qui n'existe pas cause une erreur
  - On peut l'éviter en incluant une condition
  - `DROP TABLE IF EXISTS Etudiants;`

Modification de tuples

```
UPDATE relation
SET changement [,…]
[ WHERE condition  [AND …] ];
```

- On peut modifier un tuple retourné par une requête
  - ```
    UPDATE Etudiants E
    SET E.moyenne = 3.4
    WHERE E.idul = 'CERST';
    ```
- On modifie tous les tuples qui correspondent à la recherche
  - ```
    UPDATE Etudiants E
    SET E.moyenne = 3.4
    WHERE E.age > 18;
    ```
- On peut modifier tous les tuples d'une relation
  - ```
    UPDATE Etudiants E
    SET E.moyenne = 3.4;
    ```
    ```

Suppression de tuples

```
DELETE FROM relation
[ WHERE condition [AND …] ];
```

- On peut supprimer un tuple retourné par une requête
  - `DELETE FROM Etudiants WHERE idul = 'CERST';`
- On supprime tous les tuples qui correspondent à la requête
  - `DELETE FROM Etudiants WHERE moyenne > 3.5;`
- On peut supprimer tous les tuples
  - `DELETE FROM Etudiants;`

Recherche

- Rappel de la commande de recherche

```
SELECT [DISTINCT] attribut [{INTO variable | AS nom}] [, …]
FROM relation [, …]
[ WHERE condition [AND …] ]
[ GROUP BY attribut d'agrégation ]
[ HAVING critère d'inclusion ]
[ ORDER BY attribut [DESC]]
[ combinaison deuxième_requête ] ;
```

Recherche

- On peut combiner les recherches

```
SELECT [DISTINCT] attribut [,…]
FROM relation [,…]
WHERE condition [AND …]
[ combinaison
SELECT [DISTINCT] attribut [,…]
FROM relations2 [,…]
WHERE conditions2 [AND …] ];
```

- Combinaison: UNION, EXCEPT, INTERSECT
  - Correspond à union, différence d'ensemble, intersection en algèbre relationnelle
  - Seulement UNION est disponible dans MySQL

Recherche

- Nom des étudiants qui ont suivi un cours de BD ou un cours de Prog
  - $\pi_{nom}\left(E \bowtie_{idul} S \bowtie_{sigle} \left(\sigma_{sujet=BD}(C)\right)\right) \cup \pi_{nom}\left(E \bowtie_{idul} S \bowtie_{sigle} \left(\sigma_{sujet=Prog}(C)\right)\right)$
  - ```
    SELECT E.nom
    FROM Etudiants E, Suivre S, Cours C
    WHERE E.idul=S.idul AND S.sigle=C.sigle AND C.sujet='BD'
    UNION
    SELECT E.nom
    FROM Etudiants E, Suivre S, Cours C
    WHERE E.idul=S.idul AND S.sigle=C.sigle AND C.sujet='Prog';
    ```
- Nom des étudiants qui ont suivi un cours de BD et un cours de Prog
  - $\pi_{nom}\left(E \bowtie_{idul} S \bowtie_{sigle} \left(\sigma_{sujet=BD}(C)\right)\right) \cap \pi_{nom}\left(E \bowtie_{idul} S \bowtie_{sigle} \left(\sigma_{sujet=Prog}(C)\right)\right)$
  - ```
    SELECT E.nom
    FROM Etudiants E, Suivre S, Cours C
    WHERE E.idul=S.idul AND S.sigle=C.sigle AND C.sujet='BD'
    INTERSECT
    SELECT E.nom
    FROM Etudiants E, Suivre S, Cours C
    WHERE E.idul=S.idul AND S.sigle=C.sigle AND C.sujet='Prog';
    ```

Recherche

- On peut imbriquer des recherches
  - Plus communément dans la clause WHERE, mais aussi dans FROM et HAVING

```
SELECT [DISTINCT] attribut [,…]
FROM relation [,…]
WHERE [condition [AND …]] attribut opération
(
    SELECT [DISTINCT] attribut2 [,…]
    FROM relation2 [,…]
    WHERE condition2 [AND …] );
```

Recherche

- Nom des étudiants qui ont suivi le cours GLO-2005
  - $\pi_{nom}\left(E \bowtie_{idul} \left(\sigma_{sigle=GLO-2005}(S)\right)\right)$
  - ```
    SELECT E.nom
    FROM Etudiants E, Suivre S
    WHERE E.idul=S.idul AND
          S.sigle='GLO-2005';
    ```
  - $\pi_{nom}\left(\sigma_{idul \in \pi_{idul}\left(\sigma_{sigle=GLO-2005}(S)\right)}(E)\right)$
  - ```
    SELECT E.nom
    FROM Etudiants E
    WHERE E.idul IN ( SELECT S.idul
                      FROM Suivre S
                      WHERE S.sigle='GLO-2005' );
    ```

Recherche

- Nom des étudiants qui ont suivi le cours GLO-2005
  - Autre option avec opération EXISTS
    - Retourne vrai si la requête retourne au moins un tuple
  -
```
SELECT E.nom
FROM Etudiants E
WHERE EXISTS ( SELECT *
               FROM Suivre S
               WHERE S.idul=E.idul
               AND S.sigle='GLO-2005');
```
  - Noter que la relation E définie dans la requête extérieure peut être utilisée dans la requête intérieure

Recherche

- Trouver les étudiants qui ont une moyenne plus faible qu'un étudiant de 22 ans
  - ```
    SELECT E.nom
    FROM Etudiants E
    WHERE E.moyenne < ANY (
                SELECT E2.moyenne
                FROM Etudiants E2
                WHERE E2.age=22 );
    ```
  - On aurait pu redéfinir `Etudiants E` dans la requête intérieure
  - Nommer le tuple intérieur `E2` permet de clarifier les deux tuples différents et de réutiliser le tuple `E` dans la requête intérieure
    ```

Recherche

- Trouver les étudiants qui ont une moyenne plus faible qu'un étudiant de 22 ans qui porte le même nom qu'eux
  - ```
    SELECT E.nom
    FROM Etudiants E
    WHERE E.moyenne < ANY (
                SELECT E2.moyenne
                FROM Etudiants E2
                WHERE E2.age=22
                AND E2.nom=E.nom );
    ```
  - On doit nécessairement différentier `E` et `E2` pour comparer les noms des deux tuples
    ```

Recherche

- Nom des étudiants qui ont suivi un cours de BD et un cours de Prog
  - On a vu comment l'écrire avec INTERSECT
  - Mais on a vu que l'intersection n'est pas une opération de base de l'algèbre relationnelle
  - Et n'est pas disponible dans certains SGBDR comme MySQL
  - Comment l'éviter?
-
```
SELECT E.nom
FROM Etudiants E, Suivre S, Cours C
WHERE E.idul=S.idul AND S.sigle=C.sigle
AND C.sujet='BD' AND E.nom IN (
   SELECT E2.nom
   FROM Etudiants E2, Suivre S2, Cours C2
   WHERE E2.idul=S2.idul AND
   S2.sigle=C2.sigle AND C2.sujet='Prog');
```
  - On peut également écrire une requête EXCEPT avec NOT IN

Recherche

- Trouver le nom de tous les étudiants qui ont suivi tous les cours du microprogramme P
  - Division: $\pi_{nom}\left(E \bowtie_{idul} (S \div P)\right)$
  -
```
SELECT E.nom
FROM Etudiants E
WHERE NOT EXISTS ( SELECT P.sigle
    FROM microprogrammes P
    WHERE NOT EXISTS ( SELECT S.sigle
        FROM Suivre S
        WHERE P.sigle=S.sigle AND
        E.idul=S.idul ) );
```
  - Trouver le nom des étudiants pour lesquels il n'existe pas: un sigle de Cours du microprogramme pour lesquels il n'existe pas: un tuple de l'étudiant suivant le cours

Agrégation

- Opérateurs d'agrégation en SQL
- `COUNT([DISTINCT] A)` : Le nombre de valeurs [uniques] de l'attribut A
- `SUM([DISTINCT] A)` : La somme de valeurs [uniques] de l'attribut A
- `AVG([DISTINCT] A)` : La moyenne de valeurs [uniques] de l'attribut A
- `MAX(A)` : La valeur maximale de l'attribut A
- `MIN(A)` : La valeur minimale de l'attribut A

Agrégation

- Nombre d'étudiants
  - ```
    SELECT COUNT (*)
    FROM Etudiants E;
    ```
- Note moyenne des étudiants de 25 ans ou plus
  - ```
    SELECT AVG(E.moyenne)
    FROM Etudiants E
    WHERE E.age >= 25;
    ```
- Nombre d'étudiants qui ont suivi des cours
  - ```
    SELECT COUNT (DISTINCT S.idul)
    FROM Suivre S;
    ```
- Nom et moyenne de l'étudiant avec la moyenne la plus élevée
  - ```
    SELECT E.nom, MAX(E.moyenne)
    FROM Etudiants E;
    ```
- Nom de l'étudiant avec la moyenne la plus élevée
  - ```
    SELECT E.nom
    FROM Etudiants E
    WHERE E.moyenne = ( SELECT MAX(E2.moyenne)
                        FROM Etudiants E2 );
    ```
    ```

Agrégation

```
SELECT [DISTINCT] attribut [,…]
FROM relation [,…]
[ WHERE condition [AND …] ]
[ GROUP BY attribut d'agrégation ]
[ HAVING critère d'inclusion ];
```

- Agrégation des tuples selon la valeur de l'attribut d'agrégation `GROUP BY`
  - L'opération `SELECT` est appliquée sur chaque groupe séparément
  - Si omise, toute la relation est traitée comme un groupe
- La condition de `HAVING` limite la requête aux groupes qui ont le critère d'inclusion
  - Vérifié après l'application des conditions du `WHERE`

Agrégation

- Moyenne d'âge des étudiants à chaque cote
  - Excluant les étudiants de moins de 18 ans
  - Uniquement pour les cotes ayant 2 étudiants ou plus ayant plus de 18 ans
-
```
SELECT E.cote, AVG(E.age)
FROM Etudiants2 E
WHERE E.age >= 18
GROUP BY E.cote
HAVING COUNT(*) >= 2;
```

Etudiants2

idul	nom	age	cote
ALXYZ	Alice	24	A
BOUVW	Bob	27	B
CERST	Cédric	22	A
DEOPQ	Denis	15	C
ERLMN	Érica	23	B
FRIJK	Frank	16	C
GEFGH	Gérald	26	A
HECDE	Henri	17	B
IVZAB	Ivonne	25	C
JOWXY	Josée	28	D

cote	AVG(age)
A	24
B	25

Agrégation

- Moyenne d'âge des étudiants à chaque cote
  - Excluant les étudiants de moins de 18 ans
  - Uniquement pour les cotes ayant 2 étudiants ou plus de n'importe quel âge
-
```
SELECT E.cote, AVG(E.age)
FROM Etudiants2 E
WHERE E.age >= 18
GROUP BY E.cote
HAVING 2 <= (
    SELECT COUNT(*)
    FROM Etudiants2 E2
    WHERE E2.cote=E.cote );
```

idul	nom	age	cote
ALXYZ	Alice	24	A
BOUVW	Bob	27	B
CERST	Cédric	22	A
DEOPQ	Denis	15	C
ERLMN	Érica	23	B
FRIJK	Frank	16	C
GEFGH	Gérald	26	A
HECDE	Henri	17	B
IVZAB	Ivonne	25	C
JOWXY	Josée	28	D

cote	AVG(age)
A	24
B	25
C	25

Agrégation

- La cote pour laquelle la moyenne d'âge est la plus faible
-
```
SELECT E.cote,
AVG(E.age)
FROM Etudiants2 E
WHERE E.age = (
   SELECT
   MIN(AVG(E2.age))
   FROM Etudiants2 E2
   GROUP BY E2.cote)
GROUP BY E.cote;
```
  - Ne fonctionne pas!
  - SQL ne permet pas l'imbrication de fonctions d'agrégation

idul	nom	age	cote
ALXYZ	Alice	24	A
BOUVW	Bob	27	B
CERST	Cédric	22	A
DEOPQ	Denis	15	C
ERLMN	Érica	23	B
FRIJK	Frank	16	C
GEFGH	Gérald	26	A
HECDE	Henri	17	B
IVZAB	Ivonne	25	C
JOWXY	Josée	28	D

Agrégation

- La cote pour laquelle la moyenne d'âge est la plus faible
-
```
SELECT X.cote, X.avgage
FROM (
    SELECT E.cote, AVG(E.age)
    AS avgage
    FROM Etudiants2 E
    GROUP BY E.cote ) AS X
WHERE X.avgage=(
    SELECT MIN(Y.avgage) FROM
    (SELECT E.cote,
        AVG(E.age) AS avgage
        FROM Etudiants2 E
        GROUP BY E.cote ) AS Y);
```
  - Nécessite la création d'une relation temporaire

idul	nom	age	cote
ALXYZ	Alice	24	A
BOUVW	Bob	27	B
CERST	Cédric	22	A
DEOPQ	Denis	15	C
ERLMN	Érica	23	B
FRIJK	Frank	16	C
GEFGH	Gérald	26	A
HECDE	Henri	17	B
IVZAB	Ivonne	25	C
JOWXY	Josée	28	D

cote	avgage
C	18.6

Valeur nulle

- Parfois, un tuple a un attribut dont la valeur est inconnue ou non-applicable
- SQL définit la valeur `null` pour représenter ces cas
- On peut l'utiliser dans des requêtes
  - Ex.: Étudiants qui n'ont pris aucuns cours auront une moyenne nulle
  - ```
    INSERT INTO
    Etudiants(idul,nom,age,moyenne,nas)
    VALUES
    ('DIOPQ','Diane',26,null,456789012);
    ```
  - ```
    SELECT E.nom
    FROM Etudiants E
    WHERE E.moyenne IS null;
    ```

Valeur nulle

- Une opération arithmétique sur un `null` retourne un `null`
  - ```
    UPDATE Etudiants
    SET moyenne = moyenne + 1
    WHERE idul = 'DIOPQ';
    ```
  - La moyenne reste `null`
- Une opération d'agrégation ignore les `null` à moins que ce soit les seules valeurs
  - ```
    SELECT SUM(moyenne)
    FROM Etudiants;
    ```
    - Retourne la somme des valeurs non-nulles
  - ```
    SELECT SUM(moyenne)
    FROM Etudiants
    WHERE idul = 'DIOPQ';
    ```
    - Retourne `null`
    ```

Valeur nulle

- Toute comparaison avec une valeur nulle retourne `null`
  - `SELECT 5 = null;`
  - `SELECT 5 <> null;`
  - `SELECT 5 < null;`
  - `SELECT null <> null;`
  - `SELECT null = null;`
- Toute recherche ne retourne pas la valeur nulle
  - `SELECT * FROM Etudiants WHERE moyenne > 3;`
  - `SELECT * FROM Etudiants WHERE moyenne < 3;`
  - `SELECT * FROM Etudiants WHERE moyenne <> 3;`
  - `SELECT * FROM Etudiants WHERE moyenne = 3;`

Valeur nulle

- La logique en SQL est trinaire plutôt que binaire
  - Vrai, faux, nulle
  - `true, false, null`
- Les opérateurs peuvent retourner des valeurs nulles
  - `SELECT null OR true;  1`
  - `SELECT null OR false;  null`
  - `SELECT null OR null;  null`
  - `SELECT null AND true;  null`
  - `SELECT null AND false;  0`
  - `SELECT null AND null;  null`
  - `SELECT not null;  null`

Jointure

```
SELECT attributs FROM gauche
[NATURAL] [LEFT OUTER|RIGHT OUTER|INNER] JOIN
droite
[ON condition_de_jointure]
[WHERE condition [AND …]];
```

- SQL définit plusieurs types de jointures
  - On peut spécifier des conditions pour apparier les tuples des deux relations, ou faire une jointure naturelle (égalité de tous les attributs communs)
  - `INNER`: Seulement les tuples appariés sont conservés
    - Correspond à la jointure en l'algèbre relationnelle
  - `LEFT OUTER`: Tous les tuples de la relation de gauche sont conservés
  - `RIGHT OUTER`: Tous les tuples de la relation de droite sont conservés

Jointure

idul	nom	age	moyenne	nas
ALXYZ	Alice	24	4.1	123456789
BOUVW	Bob	27	3.6	234567890
CERST	Cédric	22	3.1	345678901

idul	sigle	session	note
ALXYZ	GLO-2005	H17	87
BOUVW	GLO-2005	H17	75
DEOPQ	GLO-2005	H16	88
ALXYZ	GLO-4035	A16	67

```
SELECT * FROM Étudiants E NATURAL INNER JOIN Suivre S;
```

$$E \bowtie S$$

idul	nom	age	moyenne	nas	sigle	session	note
ALXYZ	Alice	24	4.1	123456789	GLO-2005	H17	87
BOUVW	Bob	27	3.6	234567890	GLO-2005	H17	75
ALXYZ	Alice	24	4.1	123456789	GLO-4035	A16	67

Jointure

idul	nom	age	moyenne	nas
ALXYZ	Alice	24	4.1	123456789
BOUVW	Bob	27	3.6	234567890
CERST	Cédric	22	3.1	345678901

idul	sigle	session	note
ALXYZ	GLO-2005	H17	87
BOUVW	GLO-2005	H17	75
DEOPQ	GLO-2005	H16	88
ALXYZ	GLO-4035	A16	67

```
SELECT * FROM Etudiants E NATURAL LEFT OUTER JOIN Suivre S;
```

idul	nom	age	moyenne	nas	sigle	session	note
ALXYZ	Alice	24	4.1	123456789	GLO-2005	H17	87
BOUVW	Bob	27	3.6	234567890	GLO-2005	H17	75
ALXYZ	Alice	24	4.1	123456789	GLO-4035	A16	67
CERST	Cédric	22	3.1	345678901	null	null	null

Jointure

idul	nom	age	moyenne	nas
ALXYZ	Alice	24	4.1	123456789
BOUVW	Bob	27	3.6	234567890
CERST	Cédric	22	3.1	345678901

idul	sigle	session	note
ALXYZ	GLO-2005	H17	87
BOUVW	GLO-2005	H17	75
DEOPQ	GLO-2005	H16	88
ALXYZ	GLO-4035	A16	67

```
SELECT * FROM Etudiants E NATURAL RIGHT OUTER JOIN Suivre S;
```

idul	nom	age	moyenne	nas	sigle	session	note
ALXYZ	Alice	24	4.1	123456789	GLO-2005	H17	87
BOUVW	Bob	27	3.6	234567890	GLO-2005	H17	75
ALXYZ	Alice	24	4.1	123456789	GLO-4035	A16	67
DEOPQ	null	null	null	null	GLO-2005	H16	88

Jointure

idul	nom	age	moyenne
ALXYZ	Alice	24	90
BOUVW	Bob	27	87
CERST	Cédric	22	80

idul	sigle	nom	note
ALXYZ	GLO-2005	Intro BD	87
BOUVW	GLO-2005	Intro BD	75
DEOPQ	GLO-2005	Intro BD	88
ALXYZ	GLO-4035	BD avancé	67

```
SELECT * FROM Etudiants E INNER JOIN Suivre S ON E.moyenne < S.note;
```

$$E \bowtie_{moyenne<note} S$$

(idul)	nom	age	moyenne	(idul)	sigle	nom	note
BOUVW	Bob	27	87	DEOPQ	GLO-2005	Intro BD	88
CERST	Cédric	22	80	ALXYZ	GLO-2005	Intro BD	87
CERST	Cédric	22	80	DEOPQ	GLO-2005	Intro BD	88

Jointure

idul	nom	age	moyenne
ALXYZ	Alice	24	90
BOUVW	Bob	27	87
CERST	Cédric	22	80

idul	sigle	nom	note
ALXYZ	GLO-2005	Intro BD	87
BOUVW	GLO-2005	Intro BD	75
DEOPQ	GLO-2005	Intro BD	88
ALXYZ	GLO-4035	BD avancé	67

```
SELECT * FROM Etudiants E INNER JOIN Suivre S ON E.idul = S.idul;
```

$$E \bowtie_{idul} S$$

idul	nom	age	moyenne	sigle	nom	note
ALXYZ	Alice	24	90	GLO-2005	Intro BD	87
BOUVW	Bob	27	87	GLO-2005	Intro BD	75
ALXYZ	Alice	24	90	GLO-4035	BD avancé	67

Jointure

idul	nom	age	moyenne
ALXYZ	Alice	24	90
BOUVW	Bob	27	87
CERST	Cédric	22	80

idul	sigle	nom	note
ALXYZ	GLO-2005	Intro BD	87
BOUVW	GLO-2005	Intro BD	75
DEOPQ	GLO-2005	Intro BD	88
ALXYZ	GLO-4035	BD avancé	67

```
SELECT * FROM Etudiants E INNER JOIN Suivre S ON E.idul = S.idul
WHERE S.sigle = 'GLO-2005';
```

$\sigma_{sigle=GLO-2005}(E \bowtie_{idul} S)$ ou $E \bowtie_{idul} \sigma_{sigle=GLO-2005}(S)$

idul	nom	age	moyenne	sigle	nom	note
ALXYZ	Alice	24	90	GLO-2005	Intro BD	87
BOUVW	Bob	27	87	GLO-2005	Intro BD	75

Vues

- Une vue est un sous-ensemble d'une (ou plusieurs) relation

idul	nom	age	moyenne	nas
ALXYZ	Alice	24	4.1	123456789
BOUVW	Bob	27	3.6	234567890
CERST	Cédric	22	3.1	345678901

idul	sigle	session	note
ALXYZ	GLO-2005	H17	87
BOUVW	GLO-2005	H17	75
CERST	GLO-4035	H16	88
ALXYZ	GLO-4035	A16	67
BOUVW	GLO-4027	A17	93

Vue: étudiants de GLO-2005

idul	nom	note
ALXYZ	Alice	87
BOUVW	Bob	75

Vues

- Permet de présenter l'information pertinente à l'utilisateur
  - Permet de collecter cette information dans plusieurs relations
  - Ne collecte pas l'information non-pertinente que l'utilisateur ne veut pas
- Permet de masquer de l'information à l'utilisateur
  - Information confidentielle que l'utilisateur ne doit pas voir
  - L'utilisateur d'une vue n'a pas accès aux relations sous-jacentes
  - Une technique de sécurité de BD
- Permet des gains d'efficacité d'emmagasinage des informations
  - Le SGBDR emmagasine uniquement la définition de la vue
  - Les tuples des relations sous-jacentes ne sont pas dupliqués
  - Coût de calcul: la vue doit être recalculée à chaque utilisation

Vues

- Manipulée avec des commandes similaires aux relations

```
CREATE VIEW nom AS requête;
```

```
DROP VIEW nom;
```

Vues

- Exemple:
  - ```
    CREATE VIEW listeEtudiantsGlo2005
    AS SELECT E.idul, E.nom, S.note
       FROM Etudiants E, Suivre S
       WHERE E.idul = S.idul AND
             S.sigle = 'GLO-2005';
    ```
    - listeEtudiantsGLO2005 peut être utilisé comme une relation dans des requêtes
  - ```
    DROP VIEW listeEtudiantsGlo2005;
    ```
    - Les tuples sont dans les relations sous-jacentes: aucune information n'est perdue par la suppression de la vue

Fichier source

- Il peut être utile d'exécuter du code à partir d'un fichier source
  - Permet d'éditer les requêtes dans un éditeur de code
  - Permet de sauvegarder les requêtes
  - Permet de partager facilement les requêtes et les données
- Peut être fait avec la commande

```
SOURCE nom_du_fichier;
```

  - Chemin absolu avec /
  - `SOURCE C:/Users/Richy/Desktop/test.sql;`

Résumé

- Langage SQL
  - Création, modification, suppression de relations
  - Insertion, modification, suppression de tuples
  - Recherche, agrégation
  - Valeur nulle et logique trinaire
  - Jointure
  - Création, suppression de vues
  - Utilisation d'un fichier source

Résumé

- CREATE [TEMPORARY] TABLE [IF NOT EXISTS] *nom*
  { (*nom_attribut* *domaine* [*contrainte_attribut*] [,…], [*contraintes_relation* [,…]])
  | LIKE *ancien_nom* | [AS *requête*] };
- ALTER TABLE *nom* {
  ADD COLUMN *nom_attribut* *domaine* [*contrainte_attribut*]
  | ADD [INDEX|KEY] *nom propriétés*
  | DROP COLUMN *nom_attribut*
  | DROP [INDEX|KEY] *nom* };
- DROP [TEMPORARY] TABLE [IF EXISTS] *nom* ;

Résumé

- INSERT INTO *nom_relation*
  { [(*nom_attribut* [, …])]
  VALUES (*valeurs*) [, …] | *requête* } ;
- SELECT [DISTINCT] *attribut* [{INTO *variable* | AS *nom*}] [, …]
  FROM *relation* [, …]
  [WHERE *condition* [AND …]]
  [GROUP BY *attribut d'agrégation*]
  [HAVING *critère d'inclusion*]
  [ORDER BY *attribut* [DESC]]
  [*combinaison deuxième_requête*] ;

Résumé

- SELECT *attributs* FROM *gauche* [NATURAL] [LEFT OUTER|RIGHT OUTER|INNER] JOIN *droite* [ON *condition_de_jointure*] [WHERE *condition* [AND …]];
- CREATE VIEW *nom*(*attribut*[,…])AS *requête*;
- DROP VIEW *nom*;
- SOURCE *nom_du_fichier*;

Exercices

- Ramakrishnan & Gehrke
  - 5.1, 5.2, 5.3, 5.4, 5.5
- Connolly & Begg
  - 6.3, 6.5, 6.7 à 6.28, 6.32 à 6.54
  - 7.3, 7.4, 7.17, 7.26, 7.27